

Do arquivo à cena no motor

UV2 de lightmap, materiais na Unity e primeira iluminação baked

Semana 16 — O kit inteiro, montado pela primeira vez

Objetivos de hoje

Ao final da semana você será capaz de:

- Diferenciar **UV1** (textura) de **UV2** (lightmap) e por que a luz baked precisa do próprio mapa
- **Gerar UV2** de lightmap no Blender, sem sobreposição e com padding
- **Exportar** em FBX preservando os dois canais de UV
- **Configurar** materiais PBR no shader da Unity (Albedo, Normal, Metallic/Smoothness)
- **Executar** um lightmap bake simples da cena montada
- **Capturar** renders finais do kit sob a iluminação baked



Tudo que você fez em 15 semanas está nos mapas de textura. Por que o objeto aparece cinza ao importar na Unity?

De onde viemos: conteúdo pronto, motor ainda não

A Unidade IV otimizou a **produção de conteúdo** do kit.

- **Atlas** (S13) e **Trim Sheet** (S14) — menos texturas
- **ORM, compressão e resolução** (S15) — texturas mais leves
- Todo o trabalho até aqui: **Blender e 3D Coat**

DICA

Esta semana fecha a Unidade IV **transportando** esse conteúdo, pela primeira vez, para dentro de um motor de jogo real.

UV1 e UV2: duas perguntas diferentes

O mesmo objeto carrega **dois canais de UV**, com finalidades distintas.

UV1 (desde a S2)

"Que pixel de textura aparece aqui?"

Pode ter ilhas sobrepostas (atlas) ou repetidas por tiling (trim).

UV2 (novo, hoje)

"Quanta luz chega neste ponto?"

Cada ponto físico é único → **sem nenhuma sobreposição.**

Por que a sobreposição no UV2 vaza luz

Se duas ilhas do UV2 ocupam o mesmo espaço, o bake grava a **mesma luz** em regiões fisicamente distintas do objeto.

- O resultado são **manchas de luz incoerentes** (light bleeding)
- Prevenir no UV é **muito mais rápido** que diagnosticar depois do bake
- Inspecione o UV2 no UV Editor **antes** de exportar

⚠ ATENÇÃO

Padding insuficiente entre ilhas também vaza luz entre ilhas **adjacentes** — não basta não sobrepor, é preciso espaço entre elas.

Gerando UV2 no Blender

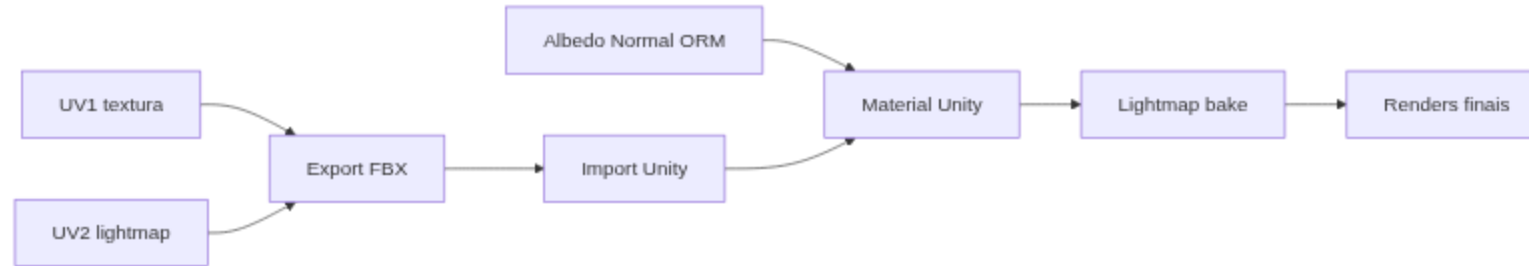
O Blender gera **automaticamente** um segundo canal de UV com regras próprias de lightmap: empacotamento sem sobreposição e com padding.

- Gerado **independente** do UV1 — não o substitui
- Ambos convivem no mesmo objeto e são exportados juntos
- Você **não reabre** o UV1 já validado desde as primeiras semanas

✓ BOAS PRÁTICAS

UV2 é um mapa **adicional**, que existe só para a luz. O UV1 continua sendo o responsável pela aparência da textura.

O caminho do asset até a cena



Os dois canais de UV vão juntos no FBX; os mapas já produzidos entram no shader da Unity; a cena marcada como *Static* recebe o bake.

Materiais na Unity: os mapas nos slots certos

O shader **Lit** (URP) espera os mapas PBR em slots próprios.

- **Base Map** ← Albedo
- **Normal Map** ← seu Normal Map
- **Metallic / Smoothness** (ou Mask Map) ← seu mapa ORM



DICA

Os mapas que você já produziu estão **certos**. O que muda é o *container* que lê esses mapas — o shader da Unity tem os próprios botões.

Cuidado 1: Smoothness é o inverso de Roughness

A Unity usa **Smoothness**; você produziu **Roughness** desde a S5. Eles são opostos.

- Superfície mais **lisa** → Smoothness **alto**
- Superfície mais **áspera** → Smoothness **baixo**
- Pode ser preciso **inverter** o canal correspondente do ORM

⊘ ERRO COMUM

Conectar Roughness como se fosse Smoothness, sem inverter, deixa a superfície mais brilhante ou mais opaca do que o material realmente é.

Cuidado 2: a flag "Normal Map" na importação

Todo Normal Map precisa ter a flag "**Normal Map**" ativada no Inspector da textura — **antes** de conectar ao material.

- Sem a flag, a Unity lê a textura como **cor comum**
- O material aceita a conexão **sem erro** — é um erro silencioso
- O resultado fica sutilmente errado: relevo **invertido ou achatado**

⚠ ATENÇÃO

Verifique sistematicamente, para cada Normal Map, se a opção está marcada no Inspector — é o primeiro ponto de checagem antes de qualquer outro diagnóstico.

Lightmap bake: da malha estática à luz calculada

Para receber o bake, o objeto precisa estar marcado como **Static** (Contribute GI).

- O bake **calcula** a iluminação indireta (luz que rebate entre superfícies)
- E a **grava como textura** no UV2 de cada objeto
- Por isso o UV2 sem sobreposição é **pré-requisito**, não detalhe

✓ BOAS PRÁTICAS

É a mesma lógica do bake de Normal e AO no Blender (S11–12): calcular uma informação complexa **uma vez** e gravá-la como textura.

Erros comuns

⊘ ERRO COMUM

Sobreposição no UV2 — vaza luz entre regiões distintas. Confira o layout no UV Editor antes de exportar.

⊘ ERRO COMUM

Normal Map sem a flag — lido como cor comum; relevo achatado ou invertido. Ative no Inspector.

⊘ ERRO COMUM

Smoothness invertido — brilho não bate com o Blender. Inverta o canal do ORM se preciso.

⊘ ERRO COMUM

Escala errada na importação — asset gigante ou minúsculo. Compare com um cubo de 1 metro.

Na indústria

Nenhum asset chega pronto ao motor. **Gerar UV2, reconfigurar materiais no shader do engine e rodar o bake** são rotina de todo pipeline — AAA ou indie.

Levar a mesma arte de uma ferramenta para outra, diagnosticando e corrigindo o que quebra no caminho, é o trabalho central de um **artista técnico** — não um obstáculo accidental.

RESUMO

Resumo

- **UV1 × UV2** — textura pode sobrepor; lightmap **nunca** sobrepõe
- **UV2** gerado automático no Blender, adicional ao UV1, exportado junto no FBX
- **Material na Unity** — Albedo, Normal (com flag) e Metallic/Smoothness nos slots
- **Smoothness** é o inverso de Roughness; **flag** obrigatória no Normal Map
- **Bake** exige objeto *Static*; o UV2 limpo é pré-requisito da luz
- **Renderers** finais do kit montado → evidência para a S17

Agora: demonstração

A seguir, ao vivo: gerar o **UV2** de um asset, exportar em FBX com os dois canais, **configurar o material** no shader Lit da Unity e rodar um **primeiro lightmap bake**.

A pergunta que você leva ao estúdio: **quais assets do meu kit já têm UV1 pronto e podem receber o UV2 agora?**

